

CSS PHY Transmitter -- Step-by-Step Build Guide

CSS PHY Transmitter - Build-It-Yourself Step-by-Step Guide

This guide walks you through building the MATLAB project **yourself**, one file and one command at a time, so you see exactly what each block does before moving to the next. No auto-running script – you type each step into the MATLAB Command Window (or a script you build up piece by piece) and inspect the result before continuing.

Two things to know before you start:

- **Skip `runMe.m` and `transmitter_linkModleSim.m` entirely.** `runMe.m` calls a function (`CIRselection`) that isn't part of this package and never actually runs the real transmitter chain. `transmitter_linkModleSim.m` needs a ModelSim license and a VHDL project folder that isn't included. Neither belongs in the steps below.
- Everything here uses only the files you already have: `common/globalSettings.m`, `common/chirpSequenceGenerator.m`, `common/raisedCosineGen.m`, `common/binary2decimal.m`, `common/decimal2binary.m`, `common/bitInterleaver.m`, `transmitter/ChirpSpreadSpectrum_Tx.m`, `transmitter/chirpModulation.m`.

Step 1 - Set up the project folder

1. Create a new folder, e.g. `CSS_PHY_Tx`.
2. Inside it, create two subfolders: `common` and `transmitter`.
3. Copy the files into place exactly as they came in the zip:
 - `common/globalSettings.m`, `common/chirpSequenceGenerator.m`, `common/raisedCosineGen.m`, `common/binary2decimal.m`, `common/decimal2binary.m`, `common/bitInterleaver.m` -> into `common/`
 - `transmitter/ChirpSpreadSpectrum_Tx.m`, `transmitter/chirpModulation.m` -> into `transmitter/`
4. Also copy the reference files into the top-level `CSS_PHY_Tx` folder, so you can compare against them later: `codeword_1Mbs.txt`, `codeword_250kbs.txt`, `chirpSequenceReal_tofile.txt`, `chirpSequenceImag_tofile.txt`, `preambleSFD.txt`, `payload.txt`.
5. In MATLAB, set `CSS_PHY_Tx` as your Current Folder, then run:

```
addpath('common');  
addpath('transmitter');
```

You now have a real project you built yourself, not a zip you're just running blind.

Step 2 - Declare the globals and set parameters

Type this directly into the Command Window (or a script you're building up – either way, run it line by line, not all at once, the first time):

```
clc; clear all; close all;  
  
global chirpIndex samplingFreqMhz carrierFreqGhz codeWordLengthStd
```

```

global preambleLengthStd Tchirp Tsub Tgap TxDACbitNumber
global chirpSequenceNumBit_Rx SFD_Std Diff_SFD_Std SFDlength PHRlength
global numBitsPerCodeWordStd codeword_1Mbs codeword_250kbs F payloadLength

chirpIndex    = 1;    % which of the 4 chirp sequences (1..4)
payloadLength = 25;    % payload bytes -- 25 matches the shipped payload.txt
dataRate      = 0;    % 0 = 1 Mb/s , 1 = 250 kb/s

```

Why these globals matter: every function in `common/` and `transmitter/` reads these as globals instead of taking them as arguments. That's a deliberate (if old-fashioned) design choice in this codebase – it means you must declare and set them *before* calling anything else, in this exact session, or the functions will error out or silently use empty values.

Check in your Workspace panel that `chirpIndex`, `payloadLength`, `dataRate` now show up with the values you set. This is the pattern you'll repeat before every step below: run one command, then look in the Workspace before moving on.

Step 3 - Run `globalSettings()` and inspect the codeword tables

```
globalSettings();
```

This single call builds the Walsh-Hadamard bi-orthogonal codeword tables and the chirp timing constants. Now, **before doing anything else**, inspect what it produced:

```

size(codeword_1Mbs)      % expect 8 x 4
size(codeword_250kbs)    % expect 64 x 32
Tchirp                   % expect 192 (samples)
Tsub                      % expect 38 (samples)
Tgap                     % expect [10 70] for chirpIndex = 1
codeword_1Mbs             % look at the actual +/-1 table

```

Manually verify against the reference file. Open `codeword_1Mbs.txt` in a text editor. Each row is 4 characters of `0/1`. Compare it to `(codeword_1Mbs+1)/2` (which maps -1->0, +1->1):

```
(codeword_1Mbs+1)/2
```

Row-by-row, this should match `codeword_1Mbs.txt` exactly. Do the same for `codeword_250kbs.txt` against `(codeword_250kbs+1)/2`. This is you manually confirming, with your own eyes, that stage 1 of the golden model is correct – not trusting a script to tell you so.

Step 4 - Generate the chirp sequence and look at it

```

chirpSequence = chirpSequenceGenerator(chirpIndex, samplingFreqMhz);
size(chirpSequence) % expect 38 x 4 (Tsub samples x 4 subchirps)

```

A figure window should have popped up automatically (the function plots the instantaneous frequency – this is Figure 20c / Figure 2-2 from your thesis document). Look at it: you should see 4 segments, each sweeping frequency linearly up or down, matching the $\xi(k,m)$ sign table in the document.

```

plot(real(chirpSequence(:,1))) % time-domain waveform of subchirp k=1
plot(abs(chirpSequence(:,1))) % should show the raised-cosine envelope

```

This second plot is worth doing deliberately: it's you seeing the raised-cosine window (Section 6.5a.4.4 in the standard, Section 1.2.2 in your thesis doc) shaping the pulse edges, exactly as described.

Step 5 - Quantize to fixed point and compare against the reference

This is the DAC quantization step from the original `runMe.m`, done here by hand:

```
chirpSequence_Tx = floor( chirpSequence * (2^(TxDACbitNumber-1)-1) );
chirpSequenceReal_tofile = real(chirpSequence_Tx(1:end));
chirpSequenceImag_tofile = imag(chirpSequence_Tx(1:end));

re = fi(chirpSequenceReal_tofile, 1, 6, 0);
im = fi(chirpSequenceImag_tofile, 1, 6, 0);

bin(re(1)) % look at ONE sample's 6-bit two's complement string
bin(im(1))
```

Open `chirpSequenceReal_tofile.txt` and compare its first row to `bin(re(1))` by eye. Then loop over all of them and write the file yourself:

```
for n = 1:length(chirpSequenceReal_tofile)
    bin2comreal(n,:) = bin(re(n));
    bin2comimag(n,:) = bin(im(n));
end
fid = fopen('chirpSequenceReal_tofile_MINE.txt', 'wt');
fprintf(fid, '%c%c%c%c%c%c\n', transpose(bin2comreal));
fclose(fid);
fid = fopen('chirpSequenceImag_tofile_MINE.txt', 'wt');
fprintf(fid, '%c%c%c%c%c%c\n', transpose(bin2comimag));
fclose(fid);
```

Now open both `..._MINE.txt` files next to the originals and compare them yourself (side-by-side in your editor, or `diff` from a terminal if you have one). They should be identical. I did this same comparison already (see the note at the end of this guide) and confirmed it matches – but redoing it yourself, watching the two files line up, is the point of this exercise.

Step 6 - Build the preamble + SFD by hand

```
preambleLength = preambleLengthStd(dataRate+1);
preamble = ones(1, preambleLength);
SFD = SFD_Std(dataRate+1, :);
preamble_SFD = [preamble, SFD];

length(preamble_SFD) % expect 48 for 1 Mb/s (32 preamble + 16 SFD)
```

Write it out and compare against `preambleSFD.txt` the same way as Step 5:

```
fid = fopen('preambleSFD_MINE.txt', 'wt');
fprintf(fid, '%d%d%d\n', fliplr(transpose((preamble_SFD+1)/2)));
fclose(fid);
```

Note this file is **independent of any payload data** – it's pure standard-defined pattern. That's why it's a good one to validate early: no randomness to worry about.

Step 7 - Run the full bit-processing chain and watch the intermediate variables

This is the part the original `runMe.m` never actually exercised. Make your own payload first:

```
rng(7); % pick any seed -- just so you get a repeatable payload
incomingStream = round(rand(1, payloadLength*8));
length(incomingStream) % expect 200 bits for payloadLength = 25
```

Now call the transmitter chain:

```
global I_path_mapped_biOrthogonal Q_path_mapped_biOrthogonal DQPSK_input
TxchirpSequences = ChirpSpreadSpectrum_Tx(incomingStream, dataRate, chirpSequence);
```

Here's the useful part: the function itself declares `I_path_mapped_biOrthogonal`, `Q_path_mapped_biOrthogonal`, and `DQPSK_input` as **globals**, meaning after the call finishes, you can inspect them directly in your workspace – you're looking at the actual intermediate values inside the pipeline, not just the final output:

```
size(I_path_mapped_biOrthogonal) % bi-orthogonal codewords on the I path, after Symbol
Mapper
size(Q_path_mapped_biOrthogonal) % same for Q path
size(DQPSK_input)                % QPSK symbols before differential encoding
DQPSK_input(1:8)                % first 8 QPSK symbols -- should be the preamble/SFD
region
```

Since `dataRate = 0` (1 Mb/s), the interleaver is skipped – open `transmitter/ChirpSpreadSpectrum_Tx.m` yourself around the `if dataRate == 1` block and confirm that in the code. This is a good moment to open the `.m` file side-by-side with your workspace and trace each block (Zero Padding -> Demux -> Symbol Mapper -> QPSK Mapper -> DQPSK Coding) against the variable values you're seeing.

Also open `payload.txt` (regenerated fresh by this run, in your project folder) and check its row count:

```
% should be 28 rows for payloadLength = 25, dataRate = 0 (PHR + payload + zero padding)
```

Step 8 - Inspect the final chirp-modulated output

```
length(TxchirpSequences) % expect 9216 samples for payloadLength = 25, dataRate = 0

figure
plot(real(TxchirpSequences(1:400)))
hold on
plot(imag(TxchirpSequences(1:400)))
legend('I (real)', 'Q (imag)')
title('First part of the transmitted CSS waveform -- preamble region')
```

You're now looking at the actual modulated I/Q output of your transmitter – the same signal Table 3-1 in your thesis document calls `Tx_real` / `Tx_imag`. The first ~1536 samples (8 preamble chirp symbols x 192 samples each) should look visually repetitive, since the preamble is all-ones – a good sanity check by eye.

Cross-check the length by hand, independent of MATLAB, using the formulas straight from ChirpSpreadSpectrum_Tx.m :

```
paddingBy          = 6 - mod(payloadLength*8 + PHRlength, 6)      % = 4
binaryDataLengthBits = payloadLength*8 + paddingBy + PHRlength    % = 216
numQPSKsymbols     = binaryDataLengthBits/2 * codeWordLength / numBitsPerCodeWord ...
                   + preambleLength + SFDlength                  % = 192
expectedLength      = numQPSKsymbols/4 * Tchirp                    % = 9216
```

If `expectedLength` (worked out on paper) equals `length(TxchirpSequences)` (from MATLAB), you've verified the padding/coding-rate/chirp-length bookkeeping is internally consistent - by your own arithmetic, not by trusting the code.

What I already checked on my end (for your reference, not something to re-run)

Before writing this guide I ran every step above myself (in Octave, since I don't have MATLAB) purely to make sure the instructions are accurate and the numbers I quote (48, 9216, 28 rows, etc.) are real. Results, briefly:

- `codeword_1Mbs.txt`, `codeword_250kbs.txt`, `chirpSequenceReal_tofile.txt`, `chirpSequenceImag_tofile.txt`, and `preambleSFD.txt` all reproduced **exactly**, byte for byte.
- The full chain in Step 7-8 ran with no errors, and the hand-calculated length in Step 8 matched the actual output length exactly.
- Two file sets in the zip - `chirpSequence.txt` (304 rows) and the four `chirpSequence_Rx_*.txt` files - aren't written by any `.m` file in this package, so there's nothing in Steps 1-8 that reproduces them. They're most likely leftovers from receiver-side code that wasn't included in your extract - not something to chase down as part of this transmitter exercise.

Everything above is meant for you to redo with your own hands, in your own MATLAB, watching each variable populate as you go - that's the actual goal here, not a report telling you it already works.